

Week 4: Learning Theory

CSCI 447: Machine Learning– Fall 2026

1

Upcoming Schedule

- Monday (9/21): Normal
- Wednesday (9/23): Gideon Popoola (GTA)
- Friday (9/25): Normal
- Monday (9/28): Normal
- Wednesday (9/30): Gideon Popoola (GTA)
- Friday (10/2): Dr. Jordan Schupbach, Charles Simonyi
- Monday (10/5): Dr. Jordan Schupbach

2

K-Medoids

- Representative object or member of a data set
- Different objective functions
 - K-means

$$J(\mathcal{C}) = \sum_{j=1}^K \sum_{\mathbf{x}_i \in \mathcal{C}} (\mathbf{x}_i - \mathbf{c}_j)^2$$

- K-medoids

$$J(\mathcal{C}) = \sum_{j=1}^K \sum_{\mathbf{x}_i \in \mathcal{C}} (\mathbf{x}_i - \mathbf{m}_j)^2$$



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

3

Partitioning Around Medoids (PAM)

Algorithm 10.4 K-Medoids Clustering

```

1: function KMEDOIDS-PAM( $\mathcal{D}, k$ )
2:   select  $\mathbf{m}_1, \dots, \mathbf{m}_k$  randomly
3:   repeat
4:     for all  $\mathbf{x}_i \in \mathcal{D}$  do
5:        $c \leftarrow \arg \min_{\mathbf{m}_j} d(\mathbf{x}_i, \mathbf{m}_j)$        $\triangleright d()$  is the distance between  $\mathbf{x}_i$  and  $\mathbf{m}_j$ .
6:       assign  $\mathbf{x}_i$  to the cluster  $c$ 
7:     end for
8:      $distortion_{k\text{-medoids}} = \sum_{j=1}^k \sum_{i \in \text{ownedby}(\mathbf{c}_j)} (\mathbf{x}_i - \mathbf{m}_j)^2$ 
9:     for all  $\mathbf{m}_i \in \mathbf{m}$  do
10:      for all  $\mathbf{x}_j \in \mathcal{D}$  where  $\mathbf{x}_j \notin \mathbf{m}$  do
11:        swap  $\mathbf{m}_i$  and  $\mathbf{x}_j$ 
12:         $distortion'_{k\text{-medoids}} = \sum_{j=1}^k \sum_{i \in \text{ownedby}(\mathbf{c}_j)} (\mathbf{x}_i - \mathbf{m}_j)^2$ 
13:        if  $distortion_{k\text{-medoids}} \leq distortion'_{k\text{-medoids}}$  then
14:          swap back
15:        end if
16:      end for
17:    end for
18:  until no change in  $\mathbf{m}_1, \dots, \mathbf{m}_k$ 
19:  return  $\mathbf{m}_1, \dots, \mathbf{m}_k$ 
20: end function

```



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

4

Fuzzy c-Means

- A method to create “fuzzy” clusters, where f is a level of “fuzzification” in the range $(1, \infty)$
 - $f \rightarrow 1$ makes clusters more “crisp”
 - $f \rightarrow \infty$ makes clusters more “fuzzy”
- Objective function

$$J(C) = \sum_{i=1}^n \sum_{j=1}^k w_j(\mathbf{x}_i)^f \|\mathbf{c}_j - \mathbf{x}_i\|^2$$

$$w_j(\mathbf{x}_i) = \frac{1}{\sum_{k=1}^c \left(\frac{\|\mathbf{c}_j - \mathbf{x}_i\|}{\|\mathbf{c}_k - \mathbf{x}_i\|} \right)^{2/(f-1)}}$$

$$\mathbf{c}_j = \frac{\sum_{i=1}^n w_j(\mathbf{x}_i)^f \mathbf{x}_i}{\sum_{i=1}^n w_j(\mathbf{x}_i)^f}$$

Cluster Evaluation

- Note that **clustering** $\neq$ **classification**!
- Evaluating clusters using classification metrics is **not valid**!
- Need to consider the *geometry* of the clusters found.
- The principal “preference bias” is to find clusters whereby
 - Clusters are well separated in distance
 - Members of clusters are “close” to one another
- Let $\delta(\mathbf{x}, \mathbf{x}')$ denote the proximity (i.e., similarity or distance) between two points $\mathbf{x}$ and $\mathbf{x}'$.
- Let $\mathbf{c}_i$ denote the centroid of the i th cluster; let it also represent the i th cluster.
- Cohesion: $\text{coh}(\mathbf{c}_i) = \sum_{\mathbf{x} \in \mathbf{c}_i} \delta(\mathbf{x}, \mathbf{c}_i)$.
- Separation: $\text{sep}(\mathbf{c}_i, \mathbf{c}_j) = \delta(\mathbf{c}_i, \mathbf{c}_j)$.
- Then a simple cluster evaluation is $CS(C) = \frac{\sum_{\mathbf{c} \in C} \text{coh}(\mathbf{c})}{\sum_{\mathbf{c}_i, \mathbf{c}_j \in C} \text{sep}(\mathbf{c}_i, \mathbf{c}_j)}$; smaller is better.

Cluster Sum Squared Error

- The name is deceptive due to the term “error.” The measures are similar to cohesion and separation respectively.
- Within Group Sum Squared Error (SSE)

$$SSE(\mathcal{C}) = \sum_{\mathbf{c}_i \in \mathcal{C}} \sum_{\mathbf{x}_j \in \mathbf{c}_i} (\mathbf{x}_j - \mathbf{c}_j)^2$$

- Between Group Sum Squared Error (SSB)

$$SSB(\mathcal{C}) = \sum_{\mathbf{c}_i \in \mathcal{C}} |\mathbf{c}_i| (\mathbf{c}_i - \bar{\mathbf{x}})^2$$

where $|\mathbf{c}_i|$ denotes the number of points in cluster $\mathbf{c}_i$.

Pseudo- F

- Also known as the “Calinski-Harabasz” measure, and derived from concepts of “analysis of variance.”
- Let k denote the number of clusters.
- Then

$$PF(\mathcal{C}) = \frac{SSB(\mathcal{C})/(k-1)}{SSE(\mathcal{C})/(|\mathcal{X}| - k)}.$$

- Higher scores are better.

Silhouette Coefficient

- First, for each data point $\mathbf{x}_i$, calculate the *average* distance between that point and every other point in the cluster: $a_i = \frac{1}{m_j} \sum_{\mathbf{x}_k \in c_j} \delta(\mathbf{x}_i, \mathbf{x}_k)$.
- Second, for each data point $\mathbf{x}_i$, find the minimum average distance to the points in each of the other clusters not containing $\mathbf{x}_i$: $b_i = \min_{c_j \in \mathcal{C} \setminus \{c(\mathbf{x}_i)\}} \left\{ \frac{1}{m_j} \sum_{\mathbf{x}_k \in c_j} \delta(\mathbf{x}_i, \mathbf{x}_k) \right\}$.
- Third, for each data point $\mathbf{x}_i$, calculate its silhouette coefficient: $s_i = \frac{b_i - a_i}{\max\{a_i, b_i\}}$.
- Finally, calculate the overall silhouette coefficient of the clusters: $sil(\mathcal{C}) = \frac{1}{|\mathcal{X}|} \sum_{\mathbf{x}_i \in \mathcal{X}} s_i$.
- This returns a value between -1 and $+1$.
- Ideally, want to be as close to $+1$ as possible.
 - $a_i < b_i$
 - a_i close to zero.

Rand Index

- Remember, **clustering $\neq$ classification**.
- Even so, there is a temptation to compare clusters to classes.
- More generally, there is a question of how to compare two distinct clusterings.
- Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be two clusterings. We calculate similarity of the clusters as follows:
 1. Let a be the number of pairs of points in the same clusters in $\mathcal{C}_1$ that are also in the same clusters in $\mathcal{C}_2$.
 2. Let b be the number of pairs of points in different clusters in $\mathcal{C}_1$ that are also in different clusters in $\mathcal{C}_2$.
 3. Let c be the number of pairs of points in the same clusters in $\mathcal{C}_1$ but that are in different clusters in $\mathcal{C}_2$.
 4. Let d be the number of pairs of points that are in different clusters in $\mathcal{C}_1$ but that are in the same clusters in $\mathcal{C}_2$.
- Calculate the Rand Index as $R = \frac{a+b}{a+b+c+d} = \frac{a+b}{\binom{n}{2}}$.

Adjusted Rand Index

- The base Rand Index is susceptible to spurious relationships among the data points.
- The “Adjusted” Rand Index corrects for these spurious relationships.
- Let $n_{ij} = |\mathcal{C}_i \cap \mathcal{C}_j|$, $a_i = \sum_j n_{ij}$, $b_j = \sum_i n_{ij}$. Contingency table:

	c_{11}	c_{12}	$\cdots$	c_{1r}	sums
c_{21}	n_{11}	n_{12}	$\cdots$	n_{1r}	a_1
c_{22}	n_{21}	n_{22}	$\cdots$	n_{2r}	a_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
c_{2s}	n_{s1}	n_{s2}	$\cdots$	n_{sr}	a_s
sums	b_1	b_2	$\cdots$	b_r	

- Then, $ARI(\mathcal{C}_i, \mathcal{C}_j) = \frac{\sum_{ij} \binom{n_{ij}}{2} - [\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}] / \binom{n}{2}}{\frac{1}{2} [\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}] - [\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}] / \binom{n}{2}}$.

Dunn Index

- A perennial issue in virtually all machine learning methods is hyperparameter tuning. This includes determining the correct number of clusters (for most clustering algorithms).
- The Dunn Index provides a “cluster validity” score based on the number of clusters.
- Given two clusters, c_i and c_j , and given $\delta(\cdot, \cdot)$, let $\Delta(c_i) = \max_{\mathbf{x}_j, \mathbf{x}_k \in c_i} \delta(\mathbf{x}_j, \mathbf{x}_k)$ be the diameter of cluster c_i . Then

$$Dunn(\mathbf{C}) = \min_{c_i \in \mathcal{C}} \left\{ \min_{c_j \in \mathcal{C} \setminus c_i} \frac{\delta(c_i, c_j)}{\max_{c_k \in \mathcal{C}} \Delta(c_k)} \right\}.$$

- The maximum Dunn index over different numbers of clusters can be used to determine an appropriate number of clusters.

Leader Clustering

- Ideally, we would like to adapt the number of clusters needed, rather than tune.
- The *leader clustering algorithm* considers the distance of an example from the existing clusters. If that distance exceeds a threshold (which is tunable), a new cluster is created.
- Also, if there is a cluster with a large number of examples, such that $\frac{|c_i|}{|X|} > \theta$, then the reference vectors are duplicated and perturbed and members reassigned to the new clusters.
- Finally, any cluster with $\frac{|c_i|}{|X|} < \varphi$ has its reference vector eliminated.

Version Spaces

- Hypothesis h is *consistent* with examples $\mathcal{D}$ iff $h(x) = c(x)$ for each example in $\mathcal{D}$.

$$\text{Consistent}(h, \mathcal{D}) \equiv (\forall \langle x, c(x) \rangle \in \mathcal{D}, h(x) = c(x))$$

- The version space $VS(h, \mathcal{D})$ is the subset of all hypotheses in $\mathcal{H}$ that are consistent with the training examples $\mathcal{D}$.

$$VS(h, \mathcal{D}) \equiv \{h \in \mathcal{H} | \text{Consistent}(h, \mathcal{D})\}$$

Representing Version Spaces

- Key Idea: Version spaces can be represented by only keeping track of maximally general (G) and maximally specific (S) members of $VS(h, \mathcal{D})$.
- Every member of the version space lies between these boundaries.

$$VS(h, \mathcal{D}) = \{h \in \mathcal{H} \mid (\exists s \in S)(\exists g \in G)(g \geq h \geq s)\}$$

Candidate Elimination Algorithm

- Initialize G to the set of maximally general hypotheses in $\mathcal{H}$.
- Initialize S to the set of maximally specific hypotheses in $\mathcal{H}$.
- Consider both positive and negative examples.

Candidate Elimination Algorithm

For each positive training example e , do

- Remove from G any hypothesis inconsistent with e .
- For each hypothesis s in S not consistent with e
 - Remove s from S .
 - Add to S all minimal generalizations of s such that
 - h is consistent with e and
 - some member of G is more general than h
- Remove from S any hypothesis that is more general than another hypothesis in S .

Candidate Elimination Algorithm

For each negative training example e , do

- Remove from S any hypothesis inconsistent with e .
- For each hypothesis g in G not consistent with e
 - Remove g from G .
 - Add to G all minimal specialization of g such that
 - h is consistent with e and
 - some member of S is more specific than h
- Remove from G any hypothesis that is less general than another hypothesis in G .

Example: EnjoySport

Sky	Temp	Humid	Wind	Water	Forecst	EnjoySpt
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

What is a general concept to describe this?

19

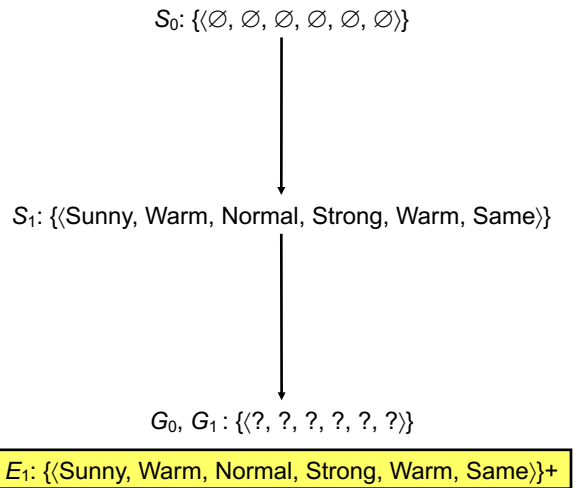
Example Trace

$S_0: \{\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle\}$

$G_0: \{\langle ?, ?, ?, ?, ?, ? \rangle\}$

20

Example Trace

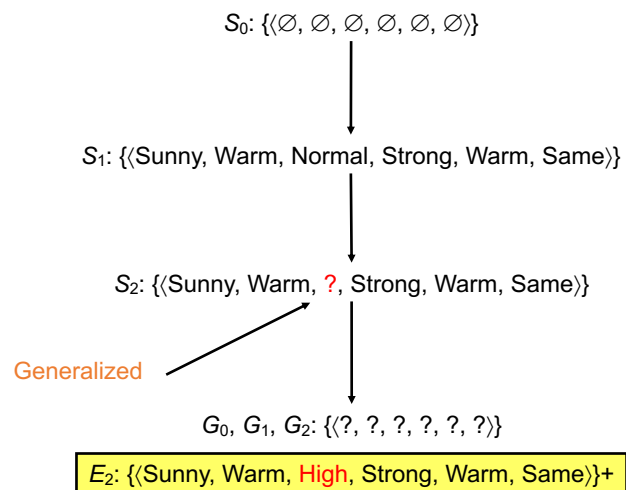


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

21

Example Trace

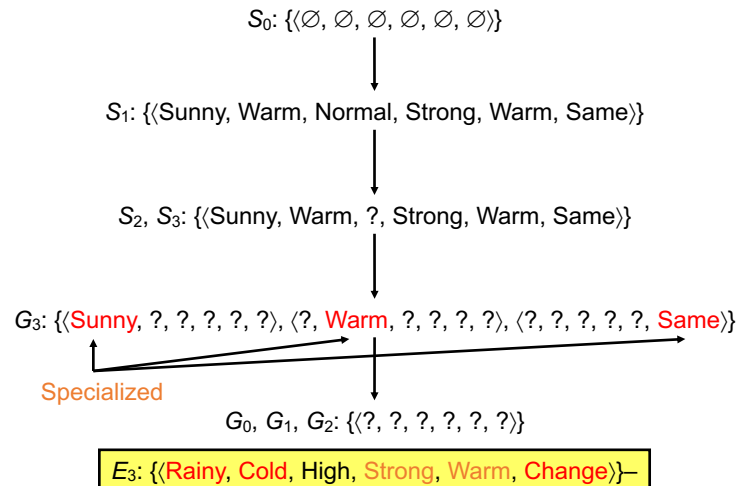


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

22

Example Trace

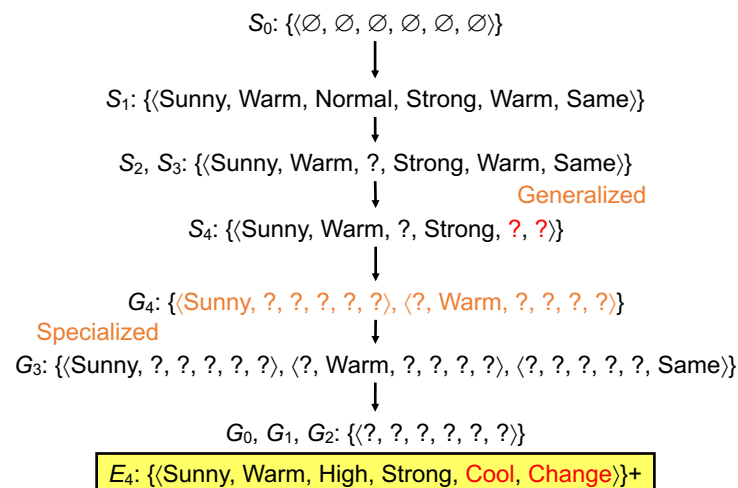


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

23

Example Trace

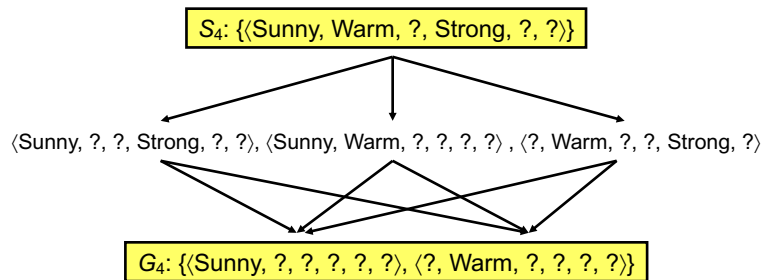


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

24

Final Version Space



Version Space Summary

- Candidate elimination will converge to the target concept if,
 - The training data $\mathcal{D}$ is error free, and
 - The target concept exists in $\mathcal{H}$.
- The algorithm is iterative—it allows choosing the training examples incrementally.
- Unseen examples can be classified from partially learned concepts.

Version Space Summary

- The approach can be used with many different languages (i.e., representations) for $\mathcal{H}$, as long as the partial order can be determined.
- Version spaces can be slow to converge, especially when encountering a large number of consistent examples.
- For some languages, the boundary sets can grow very large (exponential).
- The bias is in the representation of $\mathcal{H}$ (language).
- The candidate elimination algorithm has no preference bias since it keeps track of all consistent hypotheses.
- Note, however, that consistency entails an assumption that all examples are labeled correctly (i.e., no noise).

Computational Learning Theory

- What general laws constrain inductive learning?
- What learning problems can be solved?
- When can we trust the output of a learning algorithm?
- We seek a theory to relate:
 - Probability of successful learning
 - Number of training examples
 - Complexity of hypothesis space
 - Accuracy to which target concept is approximated
 - Manner in which training examples are presented
- Our goal is to create **Models** of learning.

Learning Models

When specifying a learning model, need to consider:

1. Who is the learner?
2. What is the problem domain?
3. Where does experience come from?
4. Is any prior knowledge available?
5. What are the performance criteria?

Typical Concept Learning Task

- Given
 - Instance space $\mathcal{X}$: possible days, each described by attributes *Sky, AirTemp, Humidity, Wind, Water, Forecast*
 - Target function $c \equiv \text{EnjoySport}: \mathcal{X} \rightarrow \{0,1\}$
 - Hypothesis $\mathcal{H}$: conjunctions of literals, e.g.,
 - $\langle ?, \text{Cold}, \text{High}, ?, ?, ? \rangle$
 - Corresponds to $[(\text{Temp}(x) = \text{Cold}) \wedge (\text{Humid}(x) = \text{High})] \Rightarrow \text{EnjoySport}(x)$
 - Training examples $\mathcal{D}$: positive and negative examples of the target function
 - $\langle \mathbf{x}_1, c(\mathbf{x}_1) \rangle, \langle \mathbf{x}_2, c(\mathbf{x}_2) \rangle, \dots, \langle \mathbf{x}_m, c(\mathbf{x}_m) \rangle$
- Determine
 - A hypothesis $h \in \mathcal{H}$ such that $h(x) = c(x), \forall x \in \mathcal{D}$ (memorize training data).
 - A hypothesis $h \in \mathcal{H}$ such that $h(x) = c(x), \forall x \in \mathcal{X}$ (generalize to unseen data).

Some Notation

- Class $\mathcal{C}$ of possible concepts
- Set of instances $\mathcal{X}$
- Length n (in attributes) of each instance
- Learner $\mathcal{L}$
- Hypothesis space $\mathcal{H}$
- Error parameter (error bound) ε
- Confidence parameter (excess error probability bound) δ
- $size(c)$ = encoding length of c , assuming some representation (also sometimes called a “description length”).

Slide 31
Copyright © 2007, Prof. John W. Sheppard



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

31

Sample Complexity

- “Sample complexity” is a measure of how many training examples are sufficient to learn the target concept. (Note, **sufficient** means having enough examples to be confident the concept has been learned, where **necessary** means the minimum number of examples required to learn the concept).
- Sample complexity depends on how we get our examples:
 - Active learning: Learner proposes instances as queries to teacher
 - Query (learner): instance $\mathbf{x}$
 - Answer (teacher): $c(\mathbf{x})$
 - Passive learning from teacher-annotated examples
 - Random process (e.g., nature) proposes examples
 - Instance $\mathbf{x}$ generated randomly, teacher provides $c(\mathbf{x})$
 - Passive learning from teacher-selected examples: Teacher (who knows c) provides examples
 - Sequence of examples (teacher): $\langle \mathbf{x}_i, c(\mathbf{x}_i) \rangle$
 - Teacher may or may not be helpful (optimal)



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

32

Active Learning

- Learner proposes instance $\mathbf{x}$
- Teacher provides $c(\mathbf{x})$
 - *Comprehensiveness*: assume c is in learner's hypothesis space $\mathcal{H}$.
 - Query is a form of inductive bias (sometimes non-trivial)
- Optimal query strategy: play 20 questions
 - Pick instance $\mathbf{x}$ such that half of hypotheses in $VS(\mathcal{H}, \mathcal{D})$ classify $\mathbf{x}$ positive, half classify $\mathbf{x}$ negative.
 - When this is possible, need $\lceil \lg |\mathcal{H}| \rceil$ queries to learn c .
 - When not possible, need even more.

Teacher Annotated

- Given
 - Set of instances $\mathcal{D}$
 - Set of hypotheses $\mathcal{H}$
 - Set of possible target concepts $\mathcal{C}$
 - Training instances generated by a fixed, unknown probability distribution $\mathcal{P}$ over $\mathcal{X}$.
- Learner observes a sequence drawn from $\mathcal{P}$
 - $\mathcal{D}$: examples of form $\langle \mathbf{x}, c(\mathbf{x}) \rangle$ for target concept c .
 - Instances $\mathbf{x} \in \mathcal{D} \sim \mathcal{P}$.
 - Teacher provides target value $c(\mathbf{x})$ for each.
- Learner must output hypothesis h estimating c
 - h evaluated on performance on subsequent instances
 - Instances still drawn according to $\mathcal{P}$

Teacher Selected

- Teacher provides training examples
 - Teacher: agent who knows c
 - Assume c is in learner's hypothesis space $\mathcal{H}$
- Optimal teacher is sometimes called a “helpful” teacher
- Optimal teaching strategy depends upon $\mathcal{H}$ used by learner

N Boolean Attributes

- Consider case: $\mathcal{H}$ = conjunctions of up to n Boolean literals and their negations.
- If n possible Boolean attributes in $\mathcal{H}$, $n + 1$ examples suffice.
- Ex: Concept = “Things that are trees”
 - Attributes = $\langle \text{Green?}, \text{Tall?}, \text{HasBark?} \rangle$
 - Begin With:
 - $\text{Green} \wedge \neg \text{Green} \wedge \text{Tall} \wedge \neg \text{Tall} \wedge \text{HasBark} \wedge \neg \text{HasBark}$
 - First positive example eliminates n literals.

N Boolean Attributes

$H_0: \text{Green} \wedge \neg\text{Green} \wedge \text{Tall} \wedge \neg\text{Tall} \wedge \text{HasBark} \wedge \neg\text{HasBark}$

$D_1: \text{Green} \wedge \text{Tall} \wedge \text{HasBark}$ (Positive example)



Revise hypothesis; eliminate contradictions. First revision eliminates n literals.

$H_1: \text{Green} \wedge \text{Tall} \wedge \text{HasBark}$

$D_2: \text{Green} \wedge \neg\text{Tall} \wedge \text{HasBark}$ (Positive example)



Revise hypothesis

$H_2: \text{Green} \wedge \text{HasBark}$



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

37

N Boolean Attributes

- Thus, a *helpful teacher* could aid learner in acquiring concept in at most $n + 1$ examples (could be less).
- Similarly, an *active learner* should be able to learn the concept in $n + 1$ examples, provided it asks the right questions.
- A learner passively receiving *randomly selected*, correctly annotated examples, may require more examples.
- *But how many?*



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

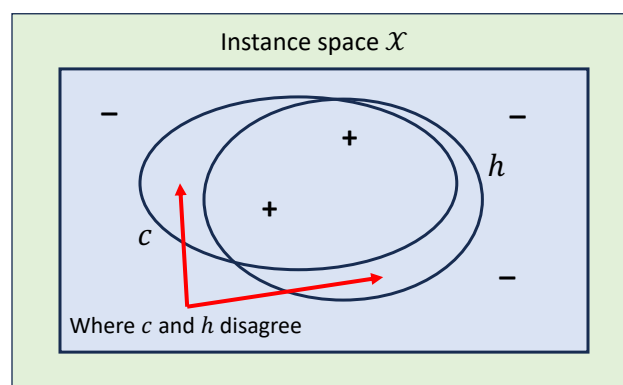
38

True Error

- **Definition:** The *true error* (denoted $error_{\mathcal{P}}(h)$) of hypothesis h w.r.t. target concept c and distribution $\mathcal{P}$ is the probability that h will misclassify an instance drawn at random according to $\mathcal{P}$.
- Two notions of error
 - *Training error* of hypothesis h w.r.t. target concept c : how often $h(x) \neq c(x)$ over *training instances* (selected according to $\mathcal{P}$).
 - *True error* of hypothesis h w.r.t. target concept c : how often $h(x) \neq c(x)$ over *future random instances*. (selected according to $\mathcal{P}$).
- Can we bound true error of h given training error of h ?

39

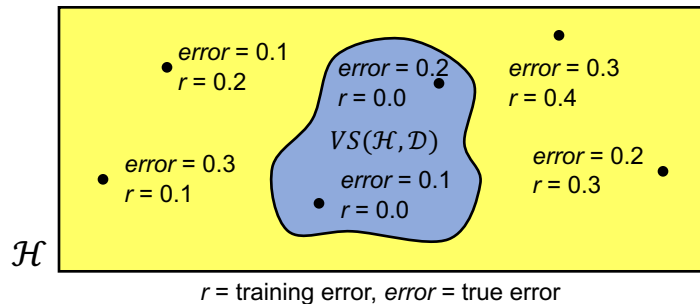
True Error



40

Exhausting the Version Space

- The version space $VS(\mathcal{H}, \mathcal{D})$ is said to be ε -exhausted w.r.t. c and $\mathcal{D}$ if every hypothesis h in $VS(\mathcal{H}, \mathcal{D})$ has true error less than ε w.r.t. c and $\mathcal{D}$.
- Denoted $\forall h \in VS(\mathcal{H}, \mathcal{D}), \text{error}_{\mathcal{P}}(h) < \varepsilon$.



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

41

Required Number of Examples

- How many examples will ε -exhaust the version space?
- Theorem** (Haussler, 1988): If the hypothesis space $\mathcal{H}$ is finite, and $\mathcal{D}$ is a sequence of $m \geq 1$ independent random examples of some target concept c , then for any $0 \leq \varepsilon \leq 1$, the probability that the version space w.r.t. $\mathcal{H}$ and $\mathcal{D}$ is not ε -exhausted (w.r.t. c) is less than or equal to $|\mathcal{H}|e^{-\varepsilon m}$.



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

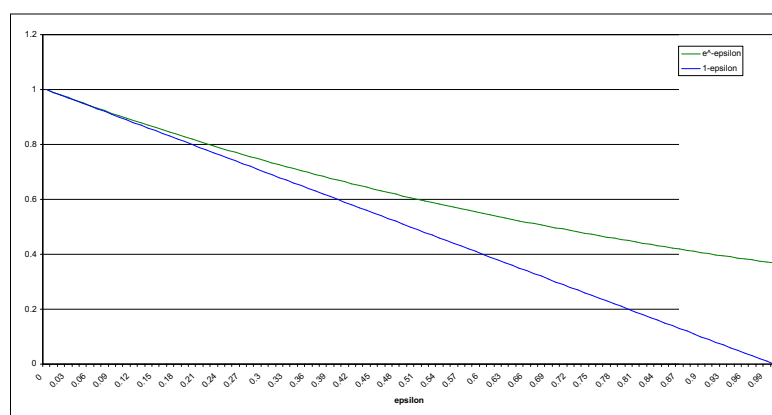
42

Haussler Theorem

- **Objective:** We want to bound the probability that there exists a “bad” hypothesis h that is consistent with m examples and satisfies $error_p(h) > \epsilon$.
- **Claim:** This probability is less than $|\mathcal{H}|e^{-\epsilon m}$.
- **Proof:**
 - Let h be such a hypothesis.
 - The probability that h is consistent with one example is less than $1 - \epsilon$.
 - Because the m examples are drawn independently of each other, the probability that h is consistent with m examples of c is less than $(1 - \epsilon)^m$.
 - The probability that some hypothesis in H is consistent with m examples of c is less than $|\mathcal{H}|(1 - \epsilon)^m$.
 - But, given $0 \leq \epsilon \leq 1$ then $(1 - \epsilon) \leq e^{-\epsilon}$.
 - Thus, $|\mathcal{H}|(1 - \epsilon)^m \leq |\mathcal{H}|e^{-\epsilon m}$. **QED.**

43

Hausler Theorem



44

Required Number of Examples

- The Haussler theorem bounds the probability that any consistent learner will output a hypothesis h with $error(h) \geq \varepsilon$.
- We want this probability to be below a specified threshold δ such that $|\mathcal{H}|e^{-\varepsilon m} \leq \delta$.
- To achieve this, solve the inequality for m where m is the number of examples required, i.e.,

$$m \geq \frac{1}{\varepsilon} \left(\ln |\mathcal{H}| + \ln \frac{1}{\delta} \right).$$

Conjunctions of Boolean Literals

- Ensure that with probability at least $1 - \delta$, every $h \in VS(\mathcal{H}, \mathcal{D})$ satisfies $error_{\mathcal{P}}(h) < \varepsilon$.
- Using Haussler's theorem, if we assume conjunctions of constraints on up to n Boolean attributes, then $|\mathcal{H}| = 3^n$.
- Thus, $m \geq \frac{1}{\varepsilon} \left(\ln 3^n + \ln \frac{1}{\delta} \right) = \frac{1}{\varepsilon} \left(n \ln 3 + \ln \left(\frac{1}{\delta} \right) \right)$.
- For EnjoySport, $|\mathcal{H}| = 729$.
- Assume our goal is probability $1 - \delta = 95\%$ s.t. $error_{\mathcal{P}}(h) < 0.1$.
- $m \geq \frac{1}{0.1} \left(\ln 729 + \ln \left(\frac{1}{0.05} \right) \right) \approx \mathbf{95.9}$.

Agnostic Learning

- So far, we have assumed $c \in \mathcal{H}$.
- For agnostic learning, we do not assume this.
- Instead, we attempt to find the closest hypothesis possible (i.e., find h that makes the fewest errors on the training set).
- Sample complexity (based on Hoeffding bounds):

$$m \geq \frac{1}{2\varepsilon^2} \left(\ln|\mathcal{H}| + \ln \frac{1}{\delta} \right).$$

PAC Learning

- Leslie Valiant proposed the theoretical framework of “probably approximately correct” (PAC) in 1984.
- The term, PAC, was coined by Dana Angluin and Philip Laird in 1986.
- C is *PAC-learnable* by $\mathcal{L}$ using $\mathcal{H}$ if for all $c \in C$, distributions $\mathcal{P}$ over $\mathcal{X}$, ε such that $0 < \varepsilon < 1/2$, and δ such that $0 < \delta < 1/2$, learner $\mathcal{L}$ will, with probability at least $(1 - \delta)$, output a hypothesis $h \in \mathcal{H}$ such that $error_{\mathcal{P}}(h) \leq \varepsilon$.
- C is *efficiently PAC-learnable* if $\mathcal{L}$ runs in time polynomial in $\frac{1}{\varepsilon}$, $\frac{1}{\delta}$, n , and $size(c)$.
- Efficiency is typically tied to sample complexity.

Ex: Unbiased Learner

- Recall sample complexity bound

$$m \geq \frac{1}{\varepsilon} \left(\ln |\mathcal{H}| + \ln \left(\frac{1}{\delta} \right) \right).$$

- Sample complexity not always polynomial.
- For unbiased learner, $|\mathcal{H}| = 2^{|\mathcal{X}|}$.
- Suppose $\mathcal{X}$ consists of n Boolean attributes.
 - $|\mathcal{X}| = 2^n$, $|\mathcal{H}| = 2^{|\mathcal{X}|}$
 - $m \geq \frac{1}{\varepsilon} \left(2^n \ln 2 + \ln \left(\frac{1}{\delta} \right) \right)$.
 - Sample complexity is exponential in n .

VC Dimension

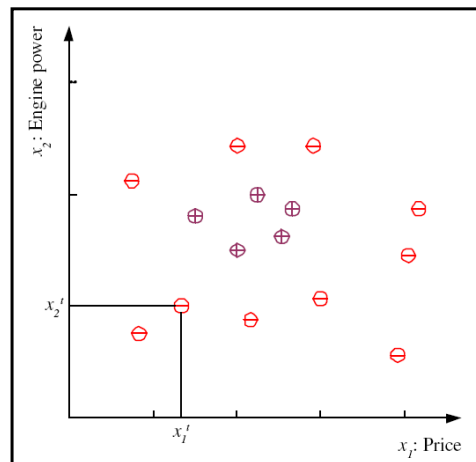
- Preceding treatment assumed finite hypothesis spaces.
- Some infinite hypothesis spaces are more expressive than other, e.g.,
 - rectangles vs. 17-sided convex polygons vs. general convex polygons.
 - linear threshold function vs. a conjunction of linear threshold units.
- Need a measure of the expressiveness of an infinite H other than its size.

Example

- Suppose we wish to classify examples of cars into “family car” or “non-family car.”
- Suppose also that the inductive bias for our classifier is a set of rectangles.
- Finally, suppose we have two attributes:
 - x_1 = price (US dollars)
 - x_2 = engine power (volume in cubic inches)

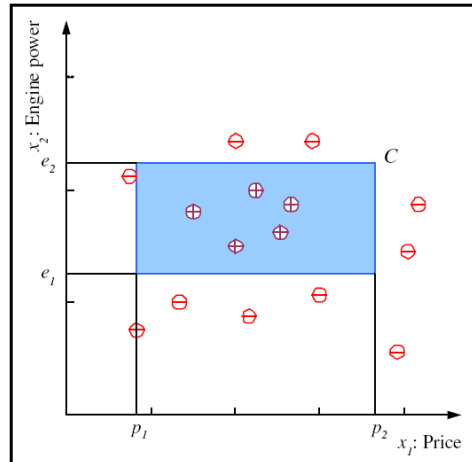
51

Training Set



52

Correct Class

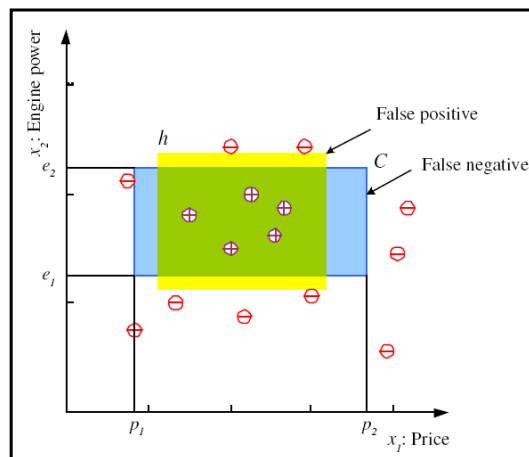


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

53

Classification Error

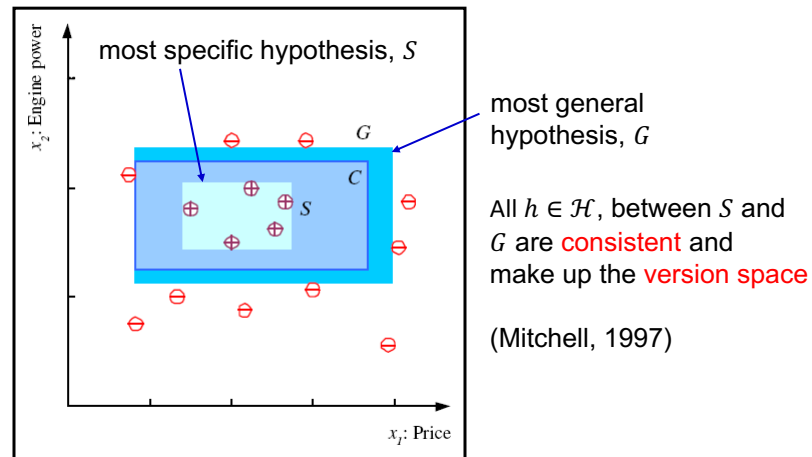


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

54

Version Space



55

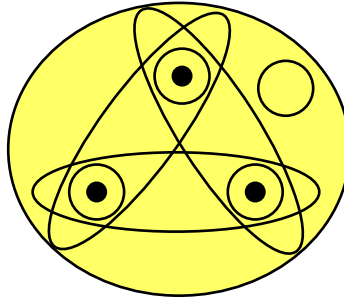
VC Dimension

- The Vapnik-Chervonenkis Dimension, denoted $VC(\mathcal{H})$,
 - Provides a measure of expressiveness of infinite hypothesis spaces
 - Is analogous to $|\mathcal{H}|$: there are bounds for sample complexity using $VC(\mathcal{H})$.
 - Depends on notion of “shattering.”
- A *partition* of a set $\mathbf{S}$ is a collection of disjoint sets $\mathbf{S}_i$ whose union is $\mathbf{S}$.
- A *dichotomy* of a set $\mathbf{S}$ is a partition of $\mathbf{S}$ into two subsets $\mathbf{S}_1$ and $\mathbf{S}_2$.
- A set of instances $\mathcal{X}$ is *shattered* by hypothesis space $\mathcal{H}$ iff for every dichotomy of $\mathcal{X}$ there exists a hypothesis of $\mathcal{H}$ consistent with this dichotomy.

56

Shattering

- N points in $\mathcal{X}$ can be labeled in 2^N ways as $+/-$.
- If there exists $h \in \mathcal{H}$ consistent for each of these, then $\mathcal{H}$ **shatters** $\mathcal{X}$.
- Then $VC(\mathcal{H}) = N$.
- Example shattering:



57

Examples: Shattered Sets

- Intervals on the real axis ($[a, b]$, $b \in \mathbb{R} > a \in \mathbb{R}$) can shatter 1 or 2 points, not 3.
- Set of any two points on X - Y plane shattered by a single line.
- Set of any three non-collinear points can be shattered by a line.
- In general, the VC dimension of linear decision surfaces in d dimensions is $d + 1$.

58

VC Dimension

- The *VC-dimension* of hypothesis space $\mathcal{H}$, $VC(\mathcal{H})$ (defined over implicit instance space $\mathcal{X}$) is the size of the largest finite subset of $\mathcal{X}$ shattered by $\mathcal{H}$.
- If arbitrarily large finite sets of $\mathcal{X}$ can be shattered by $\mathcal{H}$, then $VC(\mathcal{H}) = \infty$.
- Examples
 - $VC(\text{half intervals in } \mathbb{R}) = 1$
 - $VC(\text{intervals in } \mathbb{R}) = 2$
 - $VC(\text{half spaces in } \mathbb{R}^2) = 3$
 - $VC(\text{axis-parallel rectangles in } \mathbb{R}^2) = 4$

VC Dimension

- Using VC dimension, we can come up with a bound on sample complexity.
- Anslem Blumer *et al.* found the following upper bound (sufficient):

$$m \geq \frac{1}{\varepsilon} \left(4 \lg \frac{2}{\delta} + 8VC(\mathcal{H}) \lg \frac{13}{\varepsilon} \right).$$

- Ehrenfeucht *et al.* found the following lower bound (necessary):

$$m \geq \max \left[\frac{1}{\varepsilon} \log \frac{1}{\delta}, \frac{VC(\mathcal{H}) - 1}{32\varepsilon} \right].$$

Model Selection

- Fundamentally, learning is an *ill-posed problem*; data is not sufficient to find a unique solution
- Previously, we saw the need for an *inductive bias* corresponding to assumptions about $\mathcal{H}$.
- *Generalization*: How well a model performs on new data.
 - Overfitting: $\mathcal{H}$ more complex than C or f
 - Underfitting: $\mathcal{H}$ less complex than C or f

Triple Tradeoff

- There is a trade-off between three factors (Dietterich, 2003):
 1. Complexity of $\mathcal{H}$, $c(\mathcal{H})$,
 2. Training set size, N ,
 3. Generalization error, E , on new data
- As N increases, E decreases
- As $c(\mathcal{H})$ increases in complexity, at first E decreases, and then E increases

$VC(\mathcal{H})$ and Inductive Bias

- Unbiased hypothesis space $\mathcal{H}$ shatters the entire instance space $\mathcal{X}$ since it includes all possible hypotheses.
- In other words, $\mathcal{H}$ is able to induce every partition on set $\mathcal{X}$ of all possible instances (regardless of the set $\mathcal{X}$).
- The *larger* the subset $\mathcal{X}$ that can be shattered, the *more expressive* a hypothesis space is (i.e., the less biased).

$VC(\mathcal{H})$ and Sample Complexity

- $VC(\mathcal{H})$ prescribes an Occam algorithm for infinite hypothesis spaces.
- Given a sample $\mathcal{D}$ of m examples, find some h that is consistent with all m examples.
- If $m \geq \frac{1}{\varepsilon} \left(8VC(\mathcal{H}) \lg \frac{13}{\varepsilon} + 4 \lg \frac{2}{\delta} \right)$, then with probability at least $(1 - \delta)$, h has true error less than ε .
- If m is polynomial, it's a PAC learning algorithm.
- To be efficient, we need to produce h efficiently.
- Note: $|\mathcal{H}| > 2^m$ is required to shatter m examples (i.e., $VC(\mathcal{H}) < \lg |\mathcal{H}|$).

Making Decisions

- Much of machine learning is about making decisions.
- Suppose we are trying to decide between two options:
 - Option 1: A_1
 - Option 2: A_2
- This means we have two choices we make make:
 - Choice 1: α_1
 - Choice 2: α_2
- The goal is to use evidence $\mathbf{e}$ to make that decision.

Risk Minimization

- We consider the “cost” of a decision, denoted $\lambda(\alpha_i|A_j)$, where α_i is our choice, and the “correct” choice is A_j .
- Our goal is to minimize expected loss.
- We compute expected loss using the “risk” functional

$$R(\alpha_i|\mathbf{e}) = \sum_{j=1}^c \lambda(\alpha_i|A_j)P(A_j|\mathbf{e}).$$

- This is called a “risk” functional because it denotes the risk of making choice α_i given the evidence $\mathbf{e}$.

Computing Risk

- We begin by focusing on $P(A_j|\mathbf{e})$ and use Bayes Rule.

$$P(A_j|\mathbf{e}) = \frac{P(\mathbf{e}|A_j)P(A_j)}{P(\mathbf{e})}.$$

- Note that $P(e) = \sum_k P(\mathbf{e}|A_k)P(A_k)$.
- We seek $\alpha(e) = \operatorname{argmin}_{\alpha_i} R(\alpha_i|\mathbf{e})$.
- We denote the loss associated with choice α_i given the better choice is A_j as λ_{ij} .
- Then, assuming two choices (as an example),

$$R(\alpha_1|\mathbf{e}) = \lambda_{11}P(A_1|\mathbf{e}) + \lambda_{12}P(A_2|\mathbf{e})$$

$$R(\alpha_2|\mathbf{e}) = \lambda_{21}P(A_1|\mathbf{e}) + \lambda_{22}P(A_2|\mathbf{e})$$

- We pick A_1 over A_2 if $R(\alpha_1|\mathbf{e}) < R(\alpha_2|\mathbf{e})$.

Computing Risk

- Via some substitutions

$$\lambda_{11}P(A_1|\mathbf{e}) + \lambda_{12}P(A_2|\mathbf{e}) < \lambda_{21}P(A_1|\mathbf{e}) + \lambda_{22}P(A_2|\mathbf{e}).$$

- Rearranging terms

$$(\lambda_{11} - \lambda_{21})P(A_1|\mathbf{e}) < (\lambda_{22} - \lambda_{12})P(A_2|\mathbf{e}).$$

- Let's apply Bayes Rule again.

$$(\lambda_{11} - \lambda_{21}) \frac{P(\mathbf{e}|A_1)P(A_1)}{P(\mathbf{e})} < (\lambda_{22} - \lambda_{12}) \frac{P(\mathbf{e}|A_2)P(A_2)}{P(\mathbf{e})}.$$

- Which is equivalent to

$$(\lambda_{11} - \lambda_{21})P(\mathbf{e}|A_1)P(A_1) < (\lambda_{22} - \lambda_{12})P(\mathbf{e}|A_2)P(A_2).$$

0/1 Loss

- We apply 0/1 loss, which is among the most basic loss functions. Then
 - $\lambda_{11} = \lambda_{22} = 0$
 - $\lambda_{12} = \lambda_{21} = 1$

- Thus,

$$-P(\mathbf{e}|A_1)P(A_1) < -P(\mathbf{e}|A_2)P(A_2).$$

- From this, we choose A_1 over A_2 if

$$P(\mathbf{e}|A_1)P(A_1) > P(\mathbf{e}|A_2)P(A_2)$$

$$P(A_1|\mathbf{e}) > P(A_2|\mathbf{e}).$$

Bayes Decision Criterion

- Generally, we want the most probable hypothesis given the training data.
- **Def:** The *maximum a posteriori* hypothesis (h_{MAP}) is given as

$$\begin{aligned} h_{MAP} &= \operatorname{argmax}_{h \in \mathcal{H}} P(h|\mathcal{D}) \\ &= \operatorname{argmax}_{h \in \mathcal{H}} \frac{P(\mathcal{D}|h)}{P(\mathcal{D})} \\ &= \operatorname{argmax}_{h \in \mathcal{H}} P(\mathcal{D}|h)P(h) \end{aligned}$$

- If we assume $\forall h_i, h_j \in \mathcal{H}, i \neq j, P(h_i) = P(h_j)$, then we can simplify and choose the *maximum likelihood* hypothesis.

$$h_{ML} = \operatorname{argmax}_{h \in \mathcal{H}} P(\mathcal{D}|h)$$